

GA_Optimizer

```
import numpy as np
import copy
import random

class UniversalGeneticOptimizer:
    def __init__(self, f_eval, x_initial, bounds, y_spec, maximize_keys,
minimize_keys, equate_keys=None, max_evals=50, pop_size=10,
x_names=None):
        """
        Universal Genetic Algorithm Optimizer based on  $y = f(x)$ .
        """
        self.f_eval = f_eval
        self.x_initial = x_initial
        self.bounds = bounds
        self.y_spec = y_spec
        self.maximize_keys = maximize_keys
        self.minimize_keys = minimize_keys
        self.equate_keys = equate_keys if equate_keys else []
        self.x_names = x_names

        self.max_evals = max_evals
        self.pop_size = pop_size
        self.eval_count = 0

        # Save State Mechanism for x_opt
        self.best_loss = float('inf')
        self.x_opt = None
        self.best_y = None

    def _compute_loss(self, y_out):
        """Computes a normalized scalar penalty based on y_spec."""
        loss = 0.0
```

```

    for k in self.maximize_keys:
        if k in self.y_spec and k in y_out:
            tgt, val = self.y_spec[k], y_out[k]
            if val < tgt:
                loss += abs((tgt - val) / (abs(tgt) + 1e-9))

    for k in self.minimize_keys:
        if k in self.y_spec and k in y_out:
            tgt, val = self.y_spec[k], y_out[k]
            if val > tgt:
                loss += abs((val - tgt) / (abs(tgt) + 1e-9))

    for k in self.equate_keys:
        if k in self.y_spec and k in y_out:
            tgt, val = self.y_spec[k], y_out[k]
            # EXACT MATCH: Penalize ANY deviation (above or below)
the target
            loss += abs((val - tgt) / (abs(tgt) + 1e-9))

    return loss

def _evaluate_individual(self, x):
    """Calls the universal f(x) evaluator and computes loss."""
    eval_result = self.f_eval(x)
    y_out = eval_result[0] if isinstance(eval_result, tuple) else
eval_result

    self.eval_count += 1
    loss = self._compute_loss(y_out)

    # Save state: continuously track the best vector and print the
EXACT SPECS immediately
    if loss < self.best_loss:
        self.best_loss = loss
        self.x_opt = copy.deepcopy(x)
        self.best_y = copy.deepcopy(y_out)

    print(f"\n >>> [Sim {self.eval_count}/{self.max_evals}] NEW

```

```

BEST STATE FOUND! Loss: {self.best_loss:.4f} <<<)
    # Print every spec achieved by this new best state
    for spec_name in self.y_spec.keys():
        if spec_name in self.best_y:
            val = self.best_y[spec_name]
            tgt = self.y_spec[spec_name]
            print(f"        {spec_name:<15}: Achieved =
{val:>10.4e} | Target = {tgt:>10.4e}")
            print("    " + "-"*65)

    # Lightweight heartbeat just to show it hasn't crashed
    elif self.eval_count % 10 == 0:
        print(f"    [Sim {self.eval_count}/{self.max_evals}] Heartbeat
-> Current Best Loss: {self.best_loss:.4f}")

    return loss

def _init_population(self):
    pop = [self.x_initial]
    for _ in range(self.pop_size - 1):
        ind = [random.uniform(b[0], b[1]) for b in self.bounds]
        pop.append(ind)
    return pop

def _tournament_selection(self, pop, fitnesses, k=3):
    k = min(k, len(fitnesses))
    selected = random.sample(list(zip(pop, fitnesses)), k)
    selected.sort(key=lambda item: item[1])
    return selected[0][0]

def _crossover(self, parent1, parent2, alpha=0.5):
    child1, child2 = [], []
    for p1, p2 in zip(parent1, parent2):
        child1.append(alpha * p1 + (1 - alpha) * p2)
        child2.append((1 - alpha) * p1 + alpha * p2)
    return child1, child2

def _mutate(self, ind, mutation_rate=0.2, mutation_scale=0.1):

```

```

        mutated = []
        for i, val in enumerate(ind):
            if random.random() < mutation_rate:
                range_spread = self.bounds[i][1] - self.bounds[i][0]
                new_val = val + random.gauss(0, mutation_scale *
range_spread)
                new_val = max(self.bounds[i][0], min(new_val,
self.bounds[i][1]))
                mutated.append(new_val)
            else:
                mutated.append(val)
        return mutated

    def optimize(self):
        """Executes the universal GA loop under a strict simulation
budget."""
        print("\n" + "="*65)
        print(f" STARTING UNIVERSAL GA (Max Simulations:
{self.max_evals})")
        print("="*65)

        population = self._init_population()

        while self.eval_count < self.max_evals:
            fitnesses = []

            for x in population:
                if self.eval_count >= self.max_evals:
                    break

                loss = self._evaluate_individual(x)
                fitnesses.append(loss)

            if self.eval_count >= self.max_evals:
                break

            best_idx = np.argmin(fitnesses)
            new_population = [population[best_idx]]

```

```

        while len(new_population) < self.pop_size:
            p1 = self._tournament_selection(population, fitnesses)
            p2 = self._tournament_selection(population, fitnesses)

            c1, c2 = self._crossover(p1, p2)

            new_population.append(self._mutate(c1))
            if len(new_population) < self.pop_size:
                new_population.append(self._mutate(c2))

        population = new_population

    return self.x_opt, self.best_y

def print_report(self):
    """Prints a highly detailed final optimization report."""
    if self.x_opt is None or self.best_y is None:
        print("No optimization data to report. Run .optimize()
first.")
        return

    print("\n" + "="*65)
    print(" FINAL OPTIMIZATION REPORT")
    print("="*65)

    print("--- OPTIMAL DESIGN PARAMETERS (x_opt) ---")
    if self.x_names and len(self.x_names) == len(self.x_opt):
        for name, val in zip(self.x_names, self.x_opt):
            print(f" {name:<15}: {val:.4e}")
    else:
        for i, val in enumerate(self.x_opt):
            print(f" x[{i}]: {val:.4e}")

    print("\n--- SPECIFICATION PERFORMANCE ---")
    print(f" {'Spec':<15} | {'Target':<12} | {'Achieved':<12} |
{'Status'}")
    print(" " + "-"*58)

```

```

for spec, tgt in self.y_spec.items():
    val = self.best_y.get(spec, None)
    if val is None:
        continue

    if spec in self.maximize_keys:
        status = "PASS" if val >= tgt else "FAIL"
    elif spec in self.minimize_keys:
        status = "PASS" if val <= tgt else "FAIL"
    elif spec in self.equate_keys:
        # Exact Match Logic: Flags PASS if within 5% tolerance
        # or 1mV absolute
        error_margin = max(abs(tgt) * 0.05, 1e-3)
        status = "PASS" if abs(val - tgt) <= error_margin else
"FAIL"

    else:
        status = "INFO"

    print(f" {spec:<15} | {tgt:>12.4e} | {val:>12.4e} |
[{status}]" )
    print("="*65 + "\n")

```